

**Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии**

П Р А К Т И Ч Е С К А Я Р А Б О Т А № 1

**Сравнительный анализ фреймворков модульного тестирования для
JavaScript: Jest, Vitest и Mocha**
по дисциплине «Методы тестирования программного обеспечения»

Выполнил
студент гр. 35130903/40002

И.Р. Фёдорова

Руководитель
старший преподаватель ВШПИ

В.А. Пархоменко

СОДЕРЖАНИЕ

Введение	3
1 Методы исследования.....	4
1.1 Проблема.....	4
1.2 Критерии сравнения	4
1.3 Условия справедливого сравнения	5
1.4 Тестируемая задача: палиндромные подстроки	6
1.5 Алгоритм Манакера и его свойства.....	7
1.6 Пример и пошаговое выполнение.....	8
1.7 Порядок измерений	9
2 Разработанное программное обеспечение	10
2.1 Структура приложения	10
2.2 Работа с консолью и JSON	11
2.3 Организация одинаковых тестов	12
2.4 Функциональные и граничные проверки алгоритма	13
2.5 Тестирование JSON и интерфейса.....	14
2.6 Контрольные дефекты и качество проверок	15
2.7 Установка и запуск.....	16
3 Результаты эксперимента	17
3.1 План и входные данные	17
3.2 Среда и результаты проверки	18
3.3 Время выполнения и ресурсы	18
3.4 Сохранение и проверка результатов.....	21
4 Сравнительный анализ фреймворков.....	22
4.1 Правило выбора	22
4.2 Выбор по результатам измерений.....	23
4.3 Интерпретация различий.....	23
4.4 Ограничения и практическое применение	24
Заключение	25
Список использованных источников.....	26
Приложение 1 Структура тестов и пример проверки	27
Приложение 2 Входы и файлы эксперимента	28

ВВЕДЕНИЕ

Автоматические тесты регулярно запускаются во время разработки. Когда проверок становится много, время ожидания, загрузка процессора и потребление памяти начинают влиять на удобство работы и требования к машине. Поэтому выбор тестового фреймворка полезно обосновывать результатами на собственном приложении.

В работе сравниваются Jest, Vitest и Mocha. Цель – выбрать наиболее подходящий фреймворк для тестирования разработанного консольного приложения по времени выполнения полного набора тестов, накопленному CPU-времени и наблюдаемому пику оперативной памяти. Все три инструмента получают одинаковые входные данные и выполняют одинаковые проверки.

Приложение написано на JavaScript и реализует алгоритм Манакера. Оно принимает строку из консоли или список строк из JSON-файла. Для каждого входа выводятся максимальный палиндром, его длина и число палиндромных вхождений. Выбран один простой формат файла, чтобы код импорта и правила проверки были понятны при чтении.

Основное внимание уделяется тестированию. Подготовлены 99 отдельных тестов для каждого фреймворка: примеры с явными ожиданиями, граничные значения, независимый перебор коротких строк, свойства результата, ошибки JSON, файловый ввод и реальные запуски консоли. Дополнительно в изолированные копии программы вносятся контрольные дефекты, чтобы проверить чувствительность тестового набора.

Для сравнительного эксперимента разработан внешний наблюдатель ресурсов. Он последовательно запускает раннеры, сохраняет журналы, проверяет число пройденных тестов и записывает исходные результаты. Размер JSON-файла изменяется за счёт длины содержащихся в нём строк. На каждом размере выполняются повторные измерения, из которых рассчитываются медианы и квартили.

Первая глава описывает проблему, критерии сравнения, алгоритм и порядок эксперимента. Вторая посвящена приложению, структуре тестов и командам запуска. Третья содержит условия апробации и измеренные показатели. В четвёртой сопоставляются результаты и обосновывается выбор фреймворка с учётом границ проведённого эксперимента.

1 МЕТОДЫ ИССЛЕДОВАНИЯ

1.1 Проблема

Для одного и того же JavaScript-приложения можно использовать Jest, Vitest или Mocha. Все три инструмента запускают автоматические проверки, однако время ожидания результата и затраты компьютера могут различаться. Нужно сравнить фреймворки на одинаковом наборе тестов и выбрать наилучший по времени выполнения, процессорному времени и потреблению оперативной памяти.

В этой работе сравнение проводится на консольном приложении, реализующем алгоритм Манакера. Программа принимает строку или JSON-файл и находит самый длинный палиндром, его длину и число палиндромных вхождений. Приложение достаточно небольшое, чтобы проверить его целиком, но содержит несколько видов поведения: вычисления, проверку входных данных, чтение файлов, разбор аргументов и вывод результата.

Под наилучшим понимается фреймворк, который проходит все проверки и показывает наименьшие затраты в заданном эксперименте. Фреймворк, пропустивший тесты или завершившийся с ошибкой, не может стать победителем за счёт короткого времени запуска. Поэтому перед сравнением ресурсов проверяются корректность приложения, одинаковое количество тестов и способность набора обнаруживать ошибки.

Результат исследования должен дать ответы на четыре вопроса. Сколько времени пользователь ждёт полный запуск тестов? Сколько процессорного времени расходует этот запуск? Какой объём оперативной памяти наблюдается в его дереве процессов? Сохраняется ли порядок фреймворков при увеличении длины входных строк? Ответы опираются на реальные повторные измерения, а не на общие заявления о скорости инструментов.

1.2 Критерии сравнения

Полное время выполнения измеряется в миллисекундах внешним наблюдателем: от запуска процесса раннера до его завершения. В него входят загрузка фреймворка, обнаружение тестов, подготовка данных, сами проверки, запуск консоли в интеграционных тестах и завершение. Это время ожидания полного ре-

зультата. Внутренние таймеры раннеров не используются для основного сравнения, потому что они могут учитывать разные этапы.

CPU-время показывает, сколько времени процессоры исполняли работу обнаруженных процессов. Наблюдатель складывает пользовательское и системное время каждого процесса. Показатель выражается в миллисекундах. Он отличается от времени ожидания: процесс может ожидать чтения файла, а несколько потоков способны работать одновременно. Меньшее накопленное CPU-время означает меньшие наблюдаемые процессорные затраты данного запуска.

Оперативная память оценивается как максимальная наблюдаемая сумма RSS процессов дерева, в MiB. RSS отражает резидентные страницы процессов; это не только JavaScript-куча. Общие страницы могут учитываться несколько раз. Наблюдатель с дискретным опросом способен пропустить короткий пик. Поэтому в таблицах показатель называется наблюдаемым пиком суммы RSS, а не точным объёмом уникальной физической памяти [1].

Дополнительно рассчитывается средняя CPU-загрузка: накопленное CPU-время делится на полное время и умножается на сто. Сто процентов соответствует одному полностью занятому логическому процессору; при параллельной внутренней работе значение может превышать сто процентов. Этот процент используется для объяснения результатов, но не для выбора победителя: меньший процент сам по себе может означать более длительное ожидание.

Для каждой комбинации размера входа и фреймворка выполняются семь измерений. Основной показатель сравнения – медиана. Первый и третий квартили показывают разброс центральной половины наблюдений. Единичный удачный запуск не рассматривается как достаточное основание для рекомендации.

1.3 Условия справедливого сравнения

Все фреймворки получают одинаковый код приложения и одинаковые 99 сценариев. Для каждого раннера подготовлены четыре обычных тестовых файла. Входы, ожидаемые ответы и тела проверок совпадают. Утверждения выполняются модулем `node:assert/strict`. Предмет сравнения – запуск и исполнение одной нагрузки средствами разных раннеров [2].

Jest работает с `-runInBand` и без преобразования исходников. Mocha запускается последовательно. Vitest использует один `worker` в пуле потоков и отключённый параллелизм файлов. Все тесты синхронны, кроме действий самого консольного

приложения внутри дочерних процессов. В каждом запуске используется четыре тестовых файла и полный набор из 99 проверок [3—6].

Полный набор применяется и при обычном запуске, и при измерениях. Скрытого режима, оставляющего один пакетный тест, нет. Восемь интеграционных сценариев действительно создают процессы консольной программы; их время и наблюдаемые ресурсы входят в эксперимент. За счёт этого измеряется тестирование приложения, а не только цикл вызовов одной функции.

Coverage, watch, браузерная среда и сетевые обращения в эксперимент не включены. Они требуют другого плана измерений. Указание этих границ необходимо: результат для компактного консольного приложения не описывает автоматически большой веб-проект.

1.4 Тестируемая задача: палиндромные подстроки

Палиндромом называется строка, совпадающая со своим обращением. Для строки s требуется найти палиндромную подстроку максимальной длины и число всех непустых палиндромных вхождений. Вхождения считаются по позициям: в строке `aaa` имеются три односимвольных, два двухсимвольных и одно трёхсимвольное вхождение, всего шесть.

При нескольких максимальных ответах выбирается самый левый. Для `babad` результатом становится `bab`, хотя `aba` также имеет максимальную длину. Пустая строка даёт пустой ответ и нулевой счётчик. Строки обрабатываются с учётом регистра, без нормализации Unicode и без удаления пробелов. Поэтому визуально одинаковые последовательности с разными кодовыми точками могут считаться разными.

Прямой перебор всех подстрок с проверкой обращения прост, но при копировании и сравнении подстрок требует до $O(n^3)$ операций. Он удобен как независимый эталон для коротких входов. Для измеряемой нагрузки используется алгоритм Манакера с линейной асимптотикой [7]. Это позволяет увеличивать вход без превращения проверки корректности в чрезмерно долгий перебор.

Строка преобразуется в последовательность с разделителями. Для объяснения применяется запись

$$\text{abba} \mapsto [L, \#, a, \#, b, \#, b, \#, a, \#, R].$$

В настоящем коде L , R и разделитель являются отдельными объектами Symbol. Пользовательские символы, похожие на границы, не могут совпасть с ними. Это устраняет ошибку реализации, в которой обычные символы использовались как специальные границы.

Массив $P[i]$ хранит радиус палиндрома вокруг позиции i в преобразованной последовательности. Правый конец текущего самого далеко достигающего палиндрома обозначается R_b , его центр – C_b . Сведения о симметричной позиции позволяют не сравнивать повторно заведомо равные символы.

1.5 Алгоритм Манакера и его свойства

Рис.1.1. Поиск самого длинного палиндрома и числа вхождений

Вход: Строка s из кодовых точек

1. $t \leftarrow [L, \#, s_0, \#, \dots, s_{n-1}, \#, R]$
2. $P \leftarrow 0; C_b \leftarrow 0; R_b \leftarrow 0; b \leftarrow 0; j \leftarrow 0; z \leftarrow 0$
3. **для** $i = 1, \dots, |t| - 2$ **выполнить**
4. **если** $i < R_b$ **то**
5. $P[i] \leftarrow \min(R_b - i, P[2C_b - i])$
6. **пока** $t[i - P[i] - 1] = t[i + P[i] + 1]$ **выполнить**
7. $P[i] \leftarrow P[i] + 1$
8. **если** $i + P[i] > R_b$ **то**
9. $C_b \leftarrow i; R_b \leftarrow i + P[i]$
10. **если** $P[i] > b$ **то**
11. $b \leftarrow P[i]; j \leftarrow i$
12. $z \leftarrow z + \lfloor (P[i] + 1)/2 \rfloor$
13. $a \leftarrow \lfloor (j - b)/2 \rfloor$

Выход: $(s[a : a + b], z)$

Границы L и R различны и не встречаются внутри строки. Поэтому расширение остановится прежде, чем оба сравниваемых значения окажутся за пределами массива. Каждое успешное расширение за текущую правую границу увеличивает её. Граница не уменьшается и проходит не больше линейного числа позиций; остальная работа на позицию постоянна. Отсюда получается $O(n)$ времени и $O(n)$ дополнительной памяти.

Самый длинный ответ извлекается по радиусу и центру. Счётчик суммирует число палиндромов, соответствующих каждому центру. Строгое сравнение $P[i] > b$

сохраняет более ранний ответ при равенстве. В коде создаётся массив кодовых точек через `Array.from`, и извлечение результата выполняется из того же массива: смешение индексов UTF-16 и кодовых точек исключено.

1.6 Пример и пошаговое выполнение

Авторский пример `abba` мал и одновременно содержит чётный палиндром. В преобразованном массиве индексы от 0 до 10 соответствуют последовательности $L, \#, a, \#, b, \#, b, \#, a, \#, R$. Начальные радиусы равны нулю. Состояния после обработки каждой внутренней позиции приведены в таблице.

Таблица 1.1

Трассировка алгоритма для <code>abba</code>					
i	$P[i]$	C_b	R_b	Вклад в z	Действие
1	0	1	1	0	разделитель
2	1	2	3	1	символ <code>a</code>
3	0	2	3	0	нет расширения
4	1	4	5	1	символ <code>b</code>
5	4	5	9	2	<code>bb</code> и <code>abba</code>
6	1	5	9	1	зеркальный радиус
7	0	5	9	0	зеркальный радиус
8	1	5	9	1	радиус у границы
9	0	5	9	0	разделитель

На позиции 5 сначала совпадают два символа `b`, затем разделители, два символа `a` и внешние разделители. Получается радиус 4; следующий шаг сравнивает разные границы L и R и останавливается. Начало ответа равно $\lfloor (5 - 4)/2 \rfloor = 0$, длина равна 4. Сумма вкладов даёт $1 + 1 + 2 + 1 + 1 = 6$.

Можно проверить ответ независимо, выписав все вхождения: `a` на позиции 0, `b` на позиции 1, `b` на позиции 2, `a` на позиции 3, `bb` на позициях 1–2 и `abba` на позициях 0–3. Совпадение с таблицей проверяет и длину, и число вхождений. Радиусы всего массива равны

$[0, 0, 1, 0, 1, 4, 1, 0, 1, 0, 0]$.

1.7 Порядок измерений

Рис.1.2. Сравнение полного набора тестов

Вход: Три фреймворка, три JSON-набора, семь измерений

1. Проверить корректность приложения и контрольные дефекты
2. Сохранить версии, хеши кода и список регистрируемых тестов
3. **для каждого сочетание фреймворка и набора выполнить**
4. | Выполнить один прогрев; проверить число успешных тестов
5. **для раунд от 1 до 7 выполнить**
6. | Перемешать все сочетания с фиксированным seed
7. | **для каждого сочетание текущего раунда выполнить**
8. | | Запустить полный набор в новом процессе
9. | | **пока процесс работает выполнить**
10. | | | Обнаружить дочерние процессы, прочитав CPU и RSS
11. | | | Проверить тайм-аут и сделать паузу 10 мс
12. | | Сохранить время, ресурсы, код выхода и журнал
13. | | Проверить успешное прохождение всех 99 тестов
14. Рассчитать медианы, квартили и сравнительные места

Прогрев не включается в итоговую выборку. Каждый следующий запуск также создаёт новый процесс: состояние ЛТ предыдущего раннера не сохраняется. При этом кэши операционной системы остаются. Поэтому эксперимент не называется полностью холодным запуском.

Внутри каждого раунда все девять сочетаний выполняются по одному разу в перемешанном порядке. Фиксированный seed позволяет восстановить последовательность. Это уменьшает влияние постоянного порядка запуска, но не устраняет изменения фоновой нагрузки, температуры и частоты процессора.

Наблюдатель выполняет измерения последовательно. CPU и RSS считываются для процесса раннера и обнаруженных потомков. Процесс идентифицируется номером PID и временем создания, чтобы повторное использование PID не смешивало счётчики разных процессов. Для завершившегося потомка сохраняется последний доступный CPU-счётчик; RSS суммируется только по процессам, доступным при текущем обходе.

Из-за дискретного опроса CPU-время является нижней оценкой. Быстро созданный и завершившийся процесс может остаться незамеченным, а последний интервал до завершения может не попасть в наблюдение. Пауза 10 мс также

не гарантирует сто отсчётов в секунду: обход дерева занимает время. Для проверки качества данных сохраняются число отсчётов и максимальный реальный промежуток между ними.

Наблюдение считается успешным только при нулевом коде завершения, отсутствии тайм-аута, положительных CPU и RSS, не менее двух отсчётов и правильном числе пройденных тестов. Список ожидаемых тестов строится до серии без выполнения их тел. Фактическое число берётся из итогового сообщения каждого раннера. Такая проверка не позволяет принять пустой тестовый запуск за успешное измерение.

2 РАЗРАБОТАННОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ

2.1 Структура приложения

Приложение состоит из трёх основных модулей. `manacher.js` содержит чистую вычислительную функцию. `json-input.js` проверяет формат JSON и читает файл. `cli.js` разбирает команды, получает вход и выводит результат. Такое разделение позволяет проверять алгоритм без файловой системы, парсер без терминала, а затем всю цепочку в интеграционных тестах.

Таблица 2.1

Файлы программы и их назначение

Файл	Назначение
<code>src/manacher.js</code>	Поиск палиндрома, длины и числа вхождений
<code>src/json-input.js</code>	Единственный формат JSON и чтение файла
<code>src/cli.js</code>	Аргументы, ввод и русские сообщения
<code>src/verify.js</code>	Проверка отдельного эталонного выхода
<code>algorithm.test.js</code>	47 проверок алгоритма
<code>json.test.js</code>	25 проверок JSON и файлов
<code>cli.test.js</code>	18 проверок интерфейса
<code>workload.test.js</code>	Эталон и 8 нагрузочных тестов
<code>tests/helpers.cjs</code>	Временный файл и независимый перебор
<code>scripts/generate.js</code>	Создание входов переменной длины
<code>benchmark.py</code>	План и запуск серии измерений
<code>measurement/</code>	Замер процесса и расчёт сводки
<code>mutations/</code>	Готовые примеры трёх ошибок
<code>check_mutations.py</code>	Проверка обнаружения контрольных дефектов

JavaScript выполняется напрямую в Node.js, отдельная компиляция приложения не нужна. Рабочая программа не использует сторонних библиотек: достаточно встроенных средств Node.js. Jest, Mocha и Vitest установлены только как средства разработки. Python применяется как внешний измеритель, не входит в логику поиска палиндромов и не нужен для обычного запуска программы.

В коде используются осмысленные имена: `characters`, `longestLength`, `longestStart`, `position`. Операции обновления центра, границы и максимума записаны отдельными блоками. Комментарии объясняют, почему нужен массив кодовых точек, зачем используются уникальные границы и почему строгое сравнение сохраняет левый ответ. Они описывают причины решений, а не повторяют каждую строку программы.

2.2 Работа с консолью и JSON

Программа поддерживает небольшое число команд. Это уменьшает количество правил, которые должен помнить пользователь:

```
npm run start
npm run start -- "abba"
npm run start -- --text "abba"
npm run start -- --file data/example.json
npm run start -- --help
```

Без аргументов в интерактивном терминале программа предлагает ввести строку. При перенаправлении стандартного ввода она читает весь поток. Параметр `-text` нужен, в частности, для пустой строки или текста, похожего на параметр. Параметр `-file` принимает путь к JSON-файлу; другие форматы импорта не поддерживаются.

Единственная схема файла имеет объект с непустым массивом `strings`. Каждый его элемент – строка:

```
{"strings": ["abba", "топот", ""]}
```

Ожидаемые ответы не являются частью пользовательского входа. Программа должна вычислить их самостоятельно. Пустая строка допустима внутри массива, а пустой массив отклоняется: файл должен содержать хотя бы одну задачу. Повторяющиеся строки разрешены и обрабатываются независимо.

Парсер не удаляет пробелы и переводы строк. Регистр также сохраняется. Для строкового ввода перевод строки в перенаправленном потоке является частью

задачи. Маркер UTF-8 BOM удаляется при чтении JSON, поскольку некоторые текстовые редакторы добавляют его в начало файла. Проверка расширения не зависит от регистра, поэтому .JSON и .json допустимы.

Для abba вывод имеет вид:

```
Строка: "abba"
Самый длинный палиндром: "abba"
Длина: 4
Число палиндромных вхождений: 6
```

Строки в выводе заключаются в кавычки с помощью JSON-экранирования. Поэтому пользователь видит, где находятся пробелы, кавычки и управляющие символы. Перевод строки внутри входа не нарушает расположение подписей результата.

Неверный JSON, неправильный тип элемента, неизвестная команда и ошибка чтения файла приводят к сообщению на русском языке и коду завершения 1. Весь пакет проходит валидацию до вычисления результатов: при ошибке во втором элементе программа не успевает вывести ответ для первого. Это поведение проверяется отдельным интеграционным тестом.

2.3 Организация одинаковых тестов

В каждом из каталогов tests/jest, tests/mocha и tests/vitest находятся четыре обычных тестовых файла: алгоритм, JSON, консоль и нагрузка. Внутри видны вход, ожидаемый ответ и проверка. Фреймворк запускает эти файлы напрямую. Общий файл helpers.cjs содержит только работу с временным файлом и независимый перебор палиндромов.

Таблица 2.2

Состав полного набора для каждого фреймворка	
Группа	Число тестов
Алгоритм Манакера	47
Импорт JSON и файловая система	25
Консольное приложение	18
Отдельный эталонный файл	1
Масштабируемая нагрузка	8
Всего	99

У трёх наборов одинаковые входы, ожидаемые ответы и число повторов. Проверки записаны явно в каждом каталоге, поэтому их можно читать без перехода к функции регистрации. При изменении сценария нужно обновить его во всех трёх наборах и выполнить все тесты. Для сравнения ответов везде используется встроенный модуль Node.js `node:assert/strict`.

Тайм-аут отдельного теста равен 30 секундам. Он используется как предел отказа, а не как желательное время работы. Внешний измеритель имеет отдельный предел длительности процесса. Реальные журналы позволяют различить ошибку утверждения, некорректный вход, проблему установки и превышение времени.

2.4 Функциональные и граничные проверки алгоритма

Двадцать шесть тестов используют явно заданные ожидания. Среди них пустая строка, один символ, чётные и нечётные палиндромы, повторяющиеся символы, отсутствие длинного палиндрома, вложенные палиндромы и максимум у края строки. Для каждого случая проверяются сразу подстрока, длина и счётчик вхождений.

Например, для `abba` правильный объект содержит палиндром `abba`, длину 4 и счётчик 6. Проверка только длины была бы слабее: неверная подстрока той же длины могла бы остаться незамеченной. Проверка только непустого результата не обнаружила бы большую часть ошибок. Поэтому применяется строгое сравнение полного объекта результата.

Отдельные случаи фиксируют правила Unicode. В строке с двумя одинаковыми эмодзи длина равна двум кодовым точкам, хотя представление UTF-16 содержит больше кодовых единиц. Кириллические примеры проверяют работу с русским текстом. Символ с комбинируемым знаком не нормализуется; тест закрепляет именно такое поведение, чтобы будущая правка не изменила смысл задачи незаметно.

Шесть негативных тестов передают значения неправильного типа: `null`, `undefined`, число, логическое значение, массив и объект. Проверяется не просто наличие исключения, а его тип и понятное сообщение. Функция не должна молча приводить число к строке или принимать массив за пользовательский текст.

Девять тестов сравнивают алгоритм с независимым перебором всех двоичных строк длины от нуля до восьми. Всего рассматривается 511 строк. Медленный эталон перебирает подстроки и сравнивает их с обращением. Он не использует

радиусы или зеркальные индексы Манакера, поэтому ошибка в основном алгоритме с меньшей вероятностью повторится в проверяющем коде.

Пять тестов используют свойство сохранения числа палиндромных вхождений при обращении строки. Проверяется счётчик, а не значение левого максимума: после обращения при равных длинах левым может стать другой палиндром. Ещё один тест запускает функцию несколько раз на разных строках и подтверждает, что предыдущее вычисление не влияет на последующее.

2.5 Тестирование JSON и интерфейса

Проверки JSON охватывают правильные и неправильные документы, а также чтение реальных файлов. Положительные сценарии включают одиночный вход, пакет, пустую строку, пробелы, перевод строки, кавычки, эмодзи, повторяющиеся значения и BOM. Отрицательные – повреждённый документ, отсутствие strings, пустой пакет и элементы неправильных типов.

Каждый дефектный документ вынесен в отдельный тест. Если парсер перестанет отклонять число во втором элементе массива, отчёт раннера покажет конкретное название сценария. Один большой тест с десятком несвязанных проверок дал бы менее точную диагностику и прекратил бы выполнение после первой ошибки.

Файловые проверки создают временный каталог и удаляют его в блоке `finally`. Таким образом, тесты не зависят от заранее существующих временных файлов и не оставляют мусор после отказа. Отдельно проверяются отсутствующий файл, запрещённое расширение, регистр расширения и передача прочитанных строк вычислительной функции.

Тесты интерфейса проверяют разбор аргументов и форматирование результата как обычные функции. Восемь сценариев дополнительно запускают настоящий процесс `cli.js`. Для него проверяются код выхода, стандартный вывод и поток ошибок. Это позволяет обнаруживать дефекты точки входа, которые не видны при отдельной проверке парсера.

В дочерних запусках используются строка, пустая строка, JSON-пакет, перенаправленный поток, справка и три вида ошибок. Для успешного обычного ввода поток ошибок должен быть пустым. Для недопустимого JSON не допускается частичный успешный вывод. Тайм-аут дочернего процесса защищает набор от зависания.

2.6 Контрольные дефекты и качество проверок

Прохождение всех тестов показывает соответствие проверенным ожиданиям, но само по себе не гарантирует, что набор чувствителен к дефектам. Для дополнительной проверки подготовлены три намеренные ошибки. Готовые варианты хранятся в `mutations/rightmost.js`, `mutations/extra-count.js` и `mutations/trim-input.js`. Каждый файл содержит одну намеренную ошибку с комментарием. Скрипт копирует его во временную копию приложения и запускает тесты. Поиска и замены строк исходного кода при запуске нет.

Первая ошибка заменяет строгое сравнение максимальной длины на нестрогое. Тогда при равных длинах выбирается более поздний палиндром. Вторая добавляет лишнее вхождение на каждом центре и нарушает счётчик. Третья обрезает пробелы при чтении JSON и меняет пользовательские данные.

Таблица 2.3

Контрольные дефекты и проверяемое поведение

Дефект	Ожидаемый сигнал тестов
Правый максимум вместо левого	Нарушение ответа для <code>babad</code>
Лишние вхождения	Несовпадение счётчика для <code>abba</code>
Обрезка пробелов	Изменение строки при импорте JSON

Скрипт запускает каждый дефект под каждым раннером. Для успешной контрольной проверки требуется ненулевой код выхода, сообщение об ошибке утверждения и наличие ожидаемого сценария в журнале. Простая ошибка установки зависимостей не считается обнаруженным дефектом. Результаты сохраняются отдельно от измерений производительности.

Эта процедура представляет ограниченную проверку на трёх выбранных мутациях. Она не является полным мутационным анализом и не позволяет заявлять универсальный процент качества тестов. Также в работе не приводится неподтверждённый процент покрытия кода. Обоснование качества опирается на спецификацию, независимый эталон, классы входов, интеграционные проверки и фактически обнаруженные контрольные ошибки.

2.7 Установка и запуск

Для программы и трёх фреймворков требуется Node.js версии не ниже 22 с npm. Зависимости закреплены в `package-lock.json`. Из каталога проекта:

```
cd benchmark_app
npm ci
npm run verify
npm run demo
npm run test
```

Команда `npm run test` последовательно запускает Jest, Mocha и Vitest. Можно выполнить любой раннер отдельно:

```
npm run test:jest
npm run test:mocha
npm run test:vitest
```

Каждая команда регистрирует 99 тестов на стандартном входе. `npm run verify` сравнивает вычисленные результаты с отдельным эталонным файлом. Установка Python не требуется для этих действий.

Для замеров дополнительно требуется установленный Python не ниже 3.10. Пользователь работает только с npm-командами. При первом запуске служебный файл `scripts/python.js` автоматически создаёт окружение и устанавливает `psutil` из `requirements.txt`. Подготовка не входит в измеряемое время.

```
npm run test:quality
npm run benchmark
npm run results
```

Команда `npm run benchmark` выполняет полную серию из 63 измерений, а `npm run results` выводит таблицу последней завершённой серии. Для короткого показа доступна `npm run benchmark:quick`: три измерения на одном размере. Этот режим не заменяет полную серию. Наблюдатель создаёт новый каталог с датой и временем; существующая серия не перезаписывается. Параметры передаются после двойного дефиса в npm-команде.

Масштаб входа меняется генератором:

```
npm run generate
npm run generate -- 200 2000 20000
npm run benchmark -- --input data/size-200.json
```

В каждом файле находятся восемь реальных строк. Генератор меняет их длину, а число тестов остаётся постоянным. Стандартная серия использует длины 100, 1000

и 10000. В нагрузочных тестах ожидаемые значения рассчитываются независимыми формулами для однородных и чередующихся строк; произвольный пользовательский файл не подменяет этот специально определённый экспериментальный набор.

Для пересборки отчёта используется `My_practice.tex` с исходной преамбулой СПбПУ. Основной текст расположен в `my_folder`. Отчёт собирается pdfLaTeX, презентация – XeLaTeX, библиография – Bibex. Команда `build.ps1` из корня проекта выполняет необходимые проходы. Готовые PDF можно читать без установки LaTeX.

3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТА

3.1 План и входные данные

Эксперимент включает три размера строки: 100, 1000 и 10000 кодовых точек. В каждом JSON-файле восемь строк: четыре однородные строки из символа `a` и четыре чередующиеся строки `ab`. Восемь нагрузочных тестов регистрируются отдельно. Каждый выполняет десять вызовов алгоритма с проверкой результата.

Остальные 91 тест выполняются на каждом размере без изменения. В их состав входят 47 проверок алгоритма, 25 проверок JSON, 18 проверок консоли и одна проверка отдельного эталонного файла. Поэтому эксперимент меняет объём нагрузочных строк, сохраняя состав тестируемого приложения и полный каталог сценариев.

Таблица 3.1

Объём переменной части нагрузки

Длина строки	Строк	Вызовов в тесте	Символов за 80 вызовов
100	8	10	8000
1000	8	10	80000
10000	8	10	800000

На каждое сочетание размера и раннера приходится один исключаемый прогрев и семь измерений. Всего запланированы девять прогревов и 63 измеряемых запуска. Набор тестов выполняется полностью, включая файловые операции и про-

цессы CLI. Это отличает эксперимент от микробенчмарка одной вычислительной функции.

3.2 Среда и результаты проверки

Серия выполнена 17.09.2026. ОС Windows 11; Node.js v24.13.0; Python 3.12.14; psutil 7.0.0. Компьютер имеет 14 физических ядер, 20 логических процессоров и 31.78 GiB ОЗУ. Версии раннеров: Jest 30.0.5, Mocha 11.7.1, Vitest 3.2.4.

Успешно завершены 63 измерения и 9 прогревов. Во всех запусках пройдены 99 тестов. Каждый из трёх контрольных дефектов обнаружен каждым раннером: девять из девяти ожидаемых отказов подтверждены журналами.

Версии зависимостей закреплены для сравнимости условий. Цель не состоит в проверке самых новых версий на момент чтения отчёта. Переход на другую версию требует новой серии измерений, даже если исходный алгоритм не изменился.

3.3 Время выполнения и ресурсы

Таблица 3.2

Полное время запуска				
Длина	Раннер	Медиана, мс	Q_1	Q_3
100	Jest	1003.8	990.7	1010.9
100	Mocha	565.6	556.1	570.9
100	Vitest	1507.3	1502.9	1534.5
1000	Jest	1071.4	1053.9	1095.6
1000	Mocha	561.2	535.1	568.7
1000	Vitest	1529.1	1493.7	1542.0
10000	Jest	1481.6	1456.1	1533.3
10000	Mocha	601.0	590.6	615.6
10000	Vitest	1578.4	1558.2	1593.9

Таблица 3.3

Накопленное CPU-время, нижняя оценка				
Длина	Раннер	Медиана, мс	Q_1	Q_3
100	Jest	1046.9	984.4	1085.9
100	Mocha	453.1	414.1	484.4
100	Vitest	1671.9	1640.6	1695.3
1000	Jest	1093.8	1046.9	1109.4
1000	Mocha	453.1	437.5	476.6
1000	Vitest	1687.5	1687.5	1812.5
10000	Jest	1515.6	1484.4	1562.5
10000	Mocha	484.4	453.1	523.4
10000	Vitest	1812.5	1718.8	1875.0

Таблица 3.4

Наблюдаемый пик суммы RSS				
Длина	Раннер	Медиана, MiB	Q_1	Q_3
100	Jest	131.5	131.0	133.9
100	Mocha	100.9	100.2	101.1
100	Vitest	169.5	169.4	169.9
1000	Jest	131.9	131.5	143.3
1000	Mocha	101.0	100.7	101.0
1000	Vitest	170.1	169.7	170.2
10000	Jest	132.2	131.0	132.4
10000	Mocha	101.1	101.0	101.2
10000	Vitest	170.2	170.0	170.4

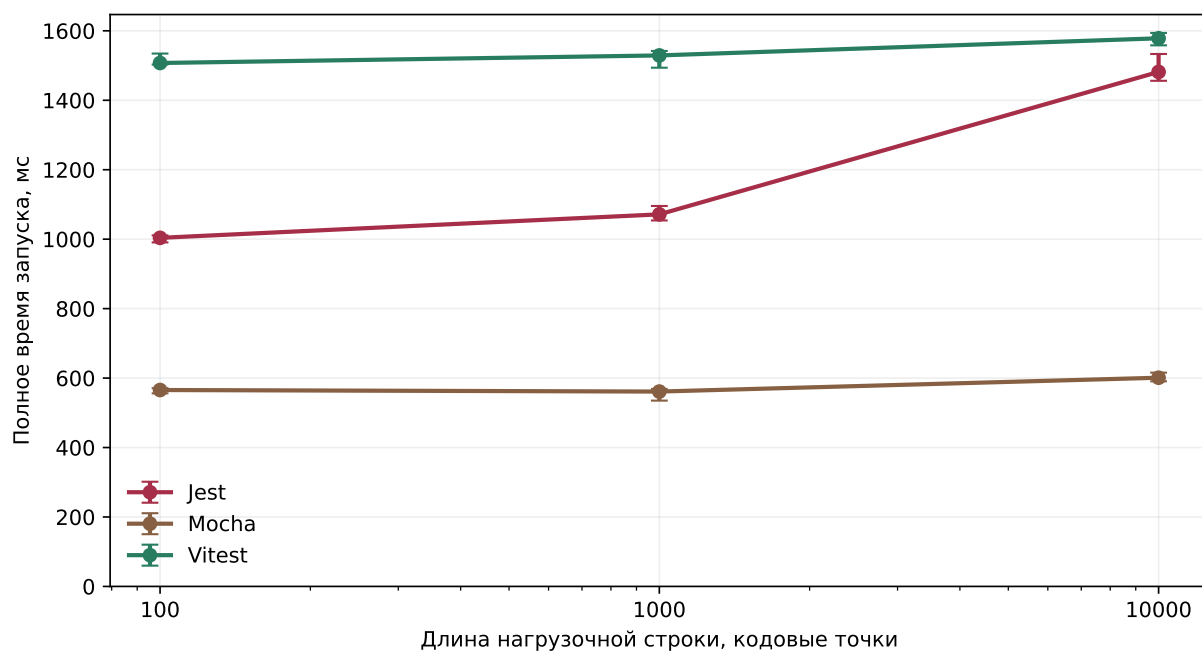


Рис.3.1. Полное время запуска. Точки – медианы, отрезки – первый и третий квартили.

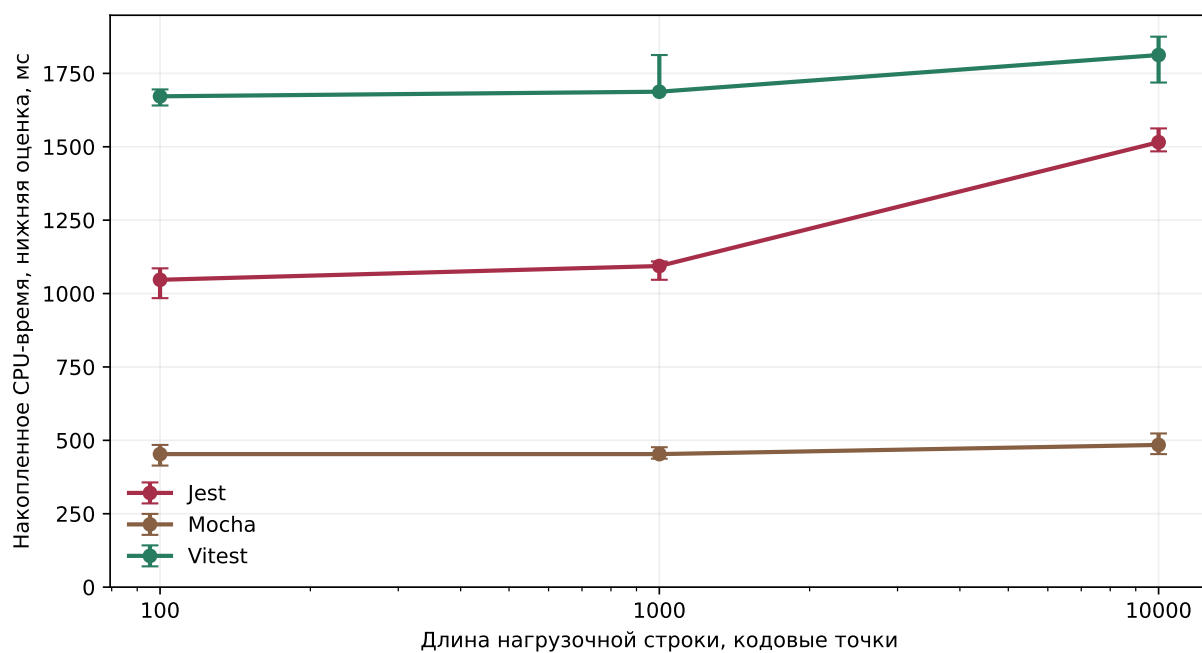


Рис.3.2. Накопленное CPU-время, нижняя оценка. Точки – медианы, отрезки – первый и третий квартили.

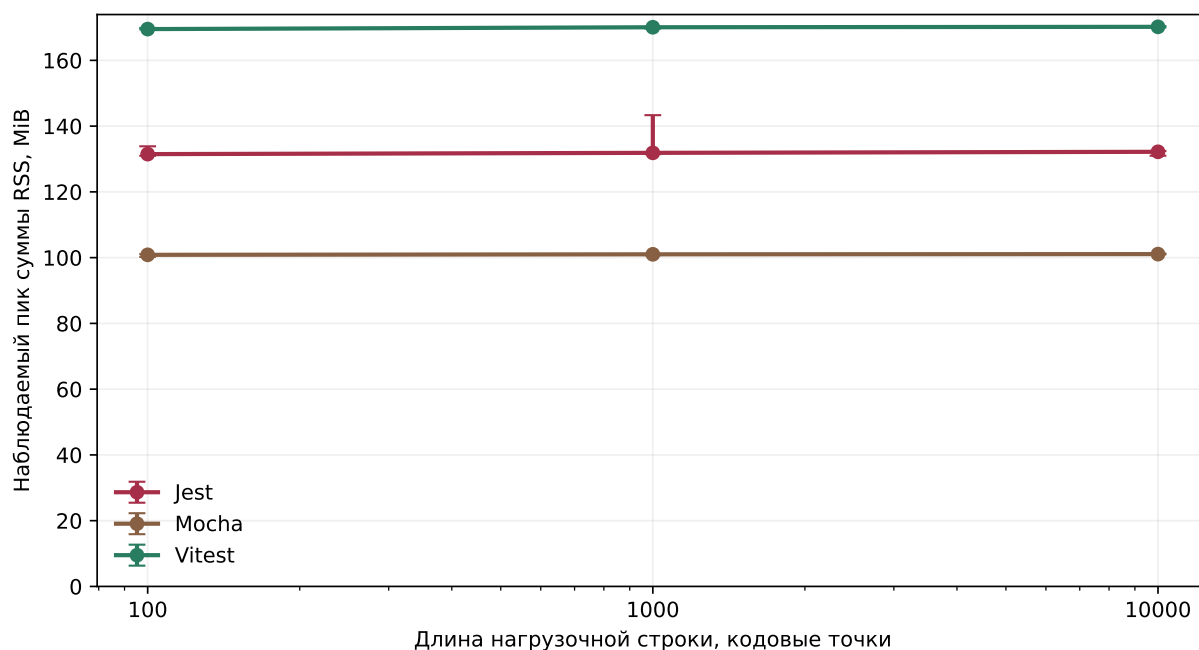


Рис.3.3. Наблюдаемый пик суммы RSS. Точки – медианы, отрезки – первый и третий квартили.

Таблица 3.5

Средняя CPU-загрузка в единицах одного логического процессора

Длина	Раннер	Медиана, %	Q_1	Q_3
100	Jest	102.8	96.2	110.4
100	Mocha	79.9	73.7	82.7
100	Vitest	110.9	108.2	111.0
1000	Jest	99.9	96.0	102.3
1000	Mocha	80.7	77.0	83.2
1000	Vitest	114.4	111.8	117.0
10000	Jest	103.7	99.7	104.7
10000	Mocha	78.5	74.2	86.2
10000	Vitest	114.2	108.8	119.5

В каждом запуске пройдены 99 тестов. Минимальное число отсчётов равно 33; максимальный интервал между обходами – 22.6 мс. Обнаружено от 9 до 14 процессов на запуск. Различие числа обнаруженных процессов также может отражать пропуск коротких процессов при опросе.

3.4 Сохранение и проверка результатов

Для каждого запуска сохраняется отдельный журнал, а исходные показатели помещаются в `raw.json`. Сводные медианы и квартили находятся в `summary.json`. Файл `tests.json` содержит имена зарегистрированных тестов по каждому вход-

ному набору. Метаданные включают версии, параметры, команды, характеристики компьютера и SHA-256 исходников, конфигураций и входов.

Таблицы и графики формируются программно из сохранённой серии. Генератор дополнительно проверяет её полноту, успешность запусков и число пройденных тестов. Старые измерения другой версии приложения не смешиваются с новой серией. Благодаря этому выводы можно связать с конкретными исходниками и набором сценариев.

Квартили вычисляются включающим методом с линейной интерполяцией между соседними отсортированными значениями. Они описывают разброс выборки, но не являются доверительными интервалами. Семь наблюдений на сочетание дают ограниченную описательную картину; статистическая значимость универсального превосходства в работе не заявляется.

4 СРАВНИТЕЛЬНЫЙ АНАЛИЗ ФРЕЙМВОРКОВ

4.1 Правило выбора

К сравнению допускаются только фреймворки, успешно выполнившие весь набор и обнаружившие выбранные контрольные дефекты. Затем на каждом из трёх размеров им присваиваются места по медиане полного времени, CPU-времени и пика RSS: меньшее значение получает лучшее место. Среднее место по девяти сочетаниям метрики и размера даёт итоговую сравнительную оценку. Все метрики и размеры имеют одинаковый вес.

Такой способ не смешивает миллисекунды с мегабайтами и делает критерий выбора явным. Он не учитывает величину разрыва между соседними местами, поэтому итог обязательно читается вместе с числовыми таблицами и квартилями. При равном среднем месте предпочтение получает меньшая медиана времени на самом большом входе. CPU-процент в рейтинг не входит.

Равные веса являются условием этого учебного сравнения, а не общим правилом для любых проектов. При жёстком ограничении памяти или при иной структуре тестов критерий нужно определить заново до измерений. Если один фреймворк оказывается первым во всех девяти сравнениях, вывод в данной серии не зависит от перераспределения положительных весов между этими показателями.

4.2 Выбор по результатам измерений

Таблица 4.1

Рейтинг по девяти сравнениям медиан		
Раннер	Среднее место	Первых мест из 9
Mocha	1.00	9
Jest	2.00	0
Vitest	3.00	0

По принятому правилу для исследуемого приложения выбран Mocha. Он получил 9 первых мест из девяти сравнений. При длине нагрузочной строки 10000 его медианы составили: 601.0 мс полного времени, 484.4 мс CPU-времени и 101.1 MiB наблюдаемого пика RSS. Выбор относится к полному набору из 99 тестов на данной машине; он не устанавливает универсального победителя для всех проектов.

4.3 Интерпретация различий

Полное время включает работу тестов и инфраструктуру раннера. Малые входы позволяют заметить постоянные затраты загрузки и организации выполнения. При увеличении длины строк возрастает доля алгоритмической нагрузки. При этом восемь запусков консольной программы остаются частью каждого измерения и формируют общую для всех фреймворков составляющую.

Для каждого фреймворка используются четыре тестовых файла: алгоритм, JSON, консоль и нагрузка. Такое разделение упрощает чтение и поиск нужной проверки. После изменения структуры вся серия выполнена заново, поскольку загрузка нескольких файлов тоже занимает время. Измерения не позволяют утверждать, что полученное соотношение сохранится при параллельном выполнении сотен модулей.

У Jest, Mocha и Vitest различаются способы подготовки среды и организации исполнения. В работе эти различия не объясняются предположениями о внутренней реализации без дополнительного профилирования. Измеренные значения показывают итоговую стоимость выбранных конфигураций. Для поиска причин конкретной задержки нужен отдельный профиль запуска.

Меньшее CPU-время не следует смешивать с меньшей средней CPU-загрузкой. Например, более долгий процесс может иметь меньший процент занятости за счёт ожидания. Аналогично меньший RSS не говорит о меньшем размере всех установленных зависимостей на диске. Каждая метрика отвечает только на свой вопрос.

4.4 Ограничения и практическое применение

CPU-счётчики и RSS считываются с паузой 10 мс. Между отсчётами могут возникать короткие процессы и пики памяти. Поэтому накопленное CPU-время является нижней оценкой, а пик памяти – наблюдаемым максимумом. Это особенно существенно для интеграционных тестов с дочерними процессами. Метод одинаков для всех раннеров, но такая одинаковость не устраняет погрешность.

Измерения выполнены на пользовательском компьютере. Фоновая нагрузка, кэши, энергосбережение и файловая система способны менять абсолютные значения. Перемешивание порядка и медианы уменьшают часть случайного влияния, но не заменяют повторение на другой машине. Результат характеризует сохранённую среду, а не все возможные условия.

Функциональность фреймворков вне данного набора не оценивается: не сравниваются удобство моков, экосистема плагинов, watch-режим, coverage и интеграция с Vite. Поэтому выбор по ресурсам относится к предмету проведённой работы. Для прикладного проекта эти дополнительные требования могут изменить итоговое решение.

Практический результат состоит в возможности повторить процедуру: установить закреплённые зависимости, запустить 99 проверок, убедиться в обнаружении контрольных дефектов, провести последовательную серию и сформировать таблицы. Если приложение развивается, сначала изменяется тестовый каталог, затем заново проверяются одинаковость нагрузки и результаты. Старые числа не переносятся на новую версию автоматически.

ЗАКЛЮЧЕНИЕ

Разработано простое консольное приложение на JavaScript с вводом строки и импортом JSON. Вычислительная функция, проверка файла и интерфейс разделены на понятные модули. Сообщения и названия тестов написаны на русском, а комментарии объясняют существенные решения в коде.

Для Jest, Mocha и Vitest подготовлен одинаковый набор из 99 отдельных тестов. Он включает функциональные, граничные, негативные, дифференциальные и интеграционные проверки. Чувствительность набора дополнительно проверена на трёх намеренных дефектах. При измерениях выполняются все тесты, включая реальные запуски консоли.

Проведена повторная серия сравнения по полному времени выполнения, CPU-времени и наблюдаемому пику RSS. Сохранены исходные замеры, журналы, список тестов, параметры и хеши файлов. Таблицы и графики создаются из этих данных автоматически.

По принятому правилу для исследуемого приложения выбран Mocha. Он получил 9 первых мест из девяти сравнений. При длине нагрузочной строки 10000 его медианы составили: 601.0 мс полного времени, 484.4 мс CPU-времени и 101.1 MiB наблюдаемого пика RSS. Выбор относится к полному набору из 99 тестов на данной машине; он не устанавливает универсального победителя для всех проектов.

Рекомендация ограничена исследуемым приложением, версиями и конфигурациями. Дальнейшее развитие сравнения может включать сбор покрытия и параллельный запуск. Для каждого такого сценария необходима новая серия на одинаковом наборе проверок.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Rodola G.* psutil documentation. — URL: <https://psutil.readthedocs.io/stable/> (дата обращения: 17.09.2026).
2. Node.js Assert documentation / Node.js. — URL: <https://nodejs.org/api/assert.html> (дата обращения: 17.09.2026).
3. Jest CLI Options / Jest. — URL: <https://jestjs.io/docs/30.0/cli> (дата обращения: 17.09.2026).
4. Configuring Jest, version 30.0 / Jest. — URL: <https://jestjs.io/docs/30.0/configuration> (дата обращения: 17.09.2026).
5. Mocha: Command-Line Usage / Mocha. — URL: <https://mochajs.org/running/cli/> (дата обращения: 17.09.2026).
6. Configuring Vitest, version 3 / Vitest. — URL: <https://v3.vitest.dev/config/> (дата обращения: 17.09.2026).
7. Manacher's Algorithm: Finding all sub-palindromes in $O(N)$ / CP-Algorithms. — URL: <https://cp-algorithms.com/string/manacher.html> (дата обращения: 17.09.2026).
8. 5. Longest Palindromic Substring / LeetCode. — URL: <https://leetcode.com/problems/longest-palindromic-substring/description/> (дата обращения: 17.09.2026).

Приложение 1

Структура тестов и пример проверки

Полный код находится в каталоге `benchmark_app`. Начать чтение удобно с `tests/jest/algorithm.test.js`. У каждого фреймворка четыре файла с одинаковыми сценариями:

```
tests/
  jest/      algorithm.test.js  json.test.js
             cli.test.js      workload.test.js
  mocha/     algorithm.test.js  json.test.js
             cli.test.js      workload.test.js
  vitest/    algorithm.test.js  json.test.js
             cli.test.js      workload.test.js
  helpers.cjs
```

Обычная проверка

Этот пример показывает проверку одного входа. В файле алгоритма аналогичные примеры перечислены в таблице, а цикл создаёт отдельный тест для каждой строки таблицы.

```
test('abba', () => {
  assert.deepEqual(manacher('abba'), {
    longest: 'abba', length: 4, count: 6,
  });
});
```

Различия запуска

Jest предоставляет `describe` и `test`. Mocha импортирует `describe` и `it` под именем `test`. Vitest явно импортирует `describe` и `test` из своего пакета. Проверки используют одинаковый `node:assert/strict`. Тайм-ауты заданы в настройках фреймворков.

Входы и файлы эксперимента

Авторский JSON

```
{"strings": ["abba", "топот", ""]}
```

Отдельный эталон

data/reference-input.json содержит строки babad и cbdd. Файл data/reference-output.json содержит ожидаемые объекты результата. Максимальные подстроки взяты из LeetCode; длины и числа вхождений проверены автором. Происхождение описано в data/SOURCES.md.

Сохранённая серия

Актуальные измерения находятся в каталоге reports приложения, в подкаталоге readable-2026-09-17. Файлы raw.json, summary.json, tests.json и metadata.json содержат исходные наблюдения, сводку, каталог тестов и параметры. Отдельные журналы находятся в logs.

Контрольные дефекты

Результаты девяти контрольных запусков находятся в файле results.json подкаталога reports/mutations; рядом сохранены журналы. Ненулевые коды выхода в этих журналах являются ожидаемым подтверждением обнаружения намеренной ошибки.